PROJECT REPORT

GesturePlay AI

An AI-based hand gesture gaming system that uses a webcam to detect hand movements and control video games without a keyboard or controller.

Project Title	GesturePlay AI (AI Hand Gesture Racing Controller)
Repository	github.com/tanmayikt98-coder/GesturePlay-AI-

1. Project Purpose

GesturePlay AI is an AI-based hand gesture gaming system. It uses a webcam to detect the player’s hand movements and converts those gestures into keyboard controls for an existing browser racing game. The primary target is Racing Limits on CrazyGames. The project does not recreate or modify the game itself; it only acts as an external controller.

Playing racing games normally requires a keyboard or gamepad. This creates a barrier for users who prefer a more natural, physical form of interaction, or who simply want a more immersive experience. GesturePlay AI solves this by turning the player’s hands into a virtual steering wheel and pedals.

The project was chosen because computer vision and real-time gesture recognition are practical and visually impressive applications of AI that third-semester students can understand and demonstrate. Target users are gamers, students learning computer vision, and anyone interested in hands-free control of applications. The main objective is to create a reliable, privacy-focused system that maps hand gestures to game controls in real time with low latency and clear safety defaults.

2. Technologies Used

The following technologies are present in the actual source code and requirements file:

• Python 3.11 — Main programming language for the entire application.
• OpenCV (opencv-python) — Webcam capture, frame processing, and on-screen overlays (landmarks, HUD).
• MediaPipe 0.10.21 — Hand landmark detection (21 points per hand) used as the core vision engine.

• NumPy — Numerical processing of landmark coordinates, angles, and smoothing filters.
• pynput — Sends keyboard events to the focused game window (only when LIVE mode is enabled).
• PyYAML — Loads configuration, game profiles, and calibration data from YAML files.
• PySide6 — Desktop graphical interface (dashboard, camera preview, calibration, profiles, and studio mode).
• qtwebview2 (Windows) — Embeds the browser game (Racing Limits) inside the application using Microsoft WebView2.
• scikit-learn and joblib — Optional machine-learning path for classifying ambiguous hand gestures.

These technologies were chosen because MediaPipe provides accurate real-time hand tracking without a GPU, OpenCV handles camera input reliably, and PySide6 allows a modern desktop studio interface. All processing runs locally on the user's machine; no cloud vision service is required. The project deliberately avoids HTML, CSS, and JavaScript for the core controller — it is a native desktop application.

3. Working and Logic

Workflow

1. The webcam captures frames (target 1280×720).
2. MediaPipe detects hand landmarks in each frame.
3. The system assigns roles to the hands (left/right grip of a virtual steering wheel).
4. Steering is calculated from the angle between the two palm centres.
5. Pedal gestures are recognised: closed fist = accelerate, open palm = brake, no hand = release.
6. Smoothing filters (including One Euro filtering) stabilise the values and reject sudden jumps.
7. A command manager converts the abstract decisions into keyboard key presses or releases.
8. In LIVE mode the keys are sent to the focused browser game; in dry-run mode they are only displayed.

User Input Processing

The only continuous input is the webcam video stream. Optional keyboard shortcuts inside the controller (Q/Esc to stop, P/S for screenshots, 1/2/3 to switch steering profiles) are also supported. Calibration is performed through a guided sequence of poses (neutral, left, right, open palm, closed fist).

Decision Making

Steering uses geometry: the angle of the line connecting the two palm centres becomes the steering input. A dead zone and response curve (linear, cubic, exponential or sigmoid) map the

angle to keyboard left/right. Pedal recognition uses simple geometric rules first (finger extension and hand openness). An optional machine-learning classifier can assist only when the pose is ambiguous. All decisions pass through hysteresis, confidence gating, and temporal stabilisation before any key is pressed.

Main Components

• CameraManager — Opens the webcam, negotiates resolution, and supplies frames.
• Hand detector (MediaPipe) — Produces 21 landmarks per detected hand.
• Steering and gesture detectors — Convert landmarks into abstract commands (steer left/right, accelerate, brake).
• Smoothing / One Euro filters — Reduce jitter while keeping fast movements responsive.
• CommandManager + KeyboardController — Own all keyboard effects; only this layer can press keys.
• PySide6 GUI / Studio Mode — Shows camera preview, gauges, calibration tools, and an embedded game window.
• Game profiles (YAML) — Map abstract commands to specific keys for different games.

Output

The system produces real-time keyboard events that control the focused racing game, an annotated camera preview with landmarks and HUD, input-state gauges, optional session recordings, and diagnostic reports (FPS, confidence, tracking losses). In dry-run mode no keys are sent; only the preview and status are shown.

4. Testing and Limitations

The repository includes a test suite that can be run with pytest. Tests use synthetic landmarks and mocked keyboard objects, so they do not require a webcam and never emit real keys. Manual testing with a real camera and the browser game is still required to verify end-to-end behaviour. Quality tools such as ruff, black, and mypy are also configured.

Challenges Observed

• Lighting, occlusion, and webcam field of view strongly affect hand tracking reliability.
• Browser games must have keyboard focus; otherwise controls do nothing.
• Laptop cameras often have a narrow field of view, forcing the player to sit farther away.

Limitations

• Gesture recognition is threshold-based and not perfectly accurate under all conditions.
• The system does not read any game telemetry (speed, RPM, gear); it only knows the controller’s own state.
• Real keyboard automation is disabled by default (dry-run). LIVE mode must be started

explicitly.
• Best results require bright, even lighting and both palms fully visible in the frame.
• The embedded game workspace relies on Microsoft WebView2 on Windows; other platforms use an external browser.

Error Handling

Camera failure, tracking loss, exceptions, window close, Q/Esc, Ctrl+C, Stop, and Emergency Stop all converge on a central key-release path so that no keys remain stuck. Missing calibration or model files cause the system to fall back to safe defaults or the rule-based detector.

Possible Future Improvements

• Broader game profile coverage and easier plugin creation.
• Improved single-hand steering modes for accessibility.
• Better automatic adaptation to different camera fields of view.
• Expanded optional ML gesture set with clearer dataset guidance.

5. GitHub and Originality

Repository Structure

The repository contains a large number of Python modules, configuration files, documentation, and scripts. Key items include:

• README.md, USER_GUIDE.md, DEVELOPER_GUIDE.md, architecture.md, controls.md, calibration.md
• Core modules: camera handling, MediaPipe hand tracking, steering and pedal detectors, smoothing, command management, keyboard controller
• GUI modules: PySide6 dashboard, studio mode, browser workspace
• Configuration: default_config.yaml, steering profiles, game profiles
• Scripts: calibrate_controller.py, check_camera.py, check_dependencies.py, training and dataset tools
• requirements.txt (and optional requirements-dev.txt for tests)

Execution Steps

1. Install 64-bit Python 3.11 on Windows 10/11.
2. Create a virtual environment and run: pip install -r requirements.txt
3. Check camera and dependencies with the provided scripts.
4. (Optional) Run the calibrator: python scripts/calibrate_controller.py --force
5. Start the GUI: python gui.py (recommended) or the CLI with --dry-run first.
6. Open or embed the racing game, give it focus, then start LIVE mode only when ready.

Individual Contributions

I contributed to the overall project development, including study and integration of the vision pipeline, gesture logic, configuration, user interface, and documentation for academic submission.

AI Assistance

AI tools were used for coding assistance, architecture understanding, and preparing documentation during development and report writing.

Learning Outcomes

• Real-time computer vision with MediaPipe and OpenCV.
• Mapping continuous sensor data to discrete game controls with smoothing and safety logic.
• Building a responsive desktop application with PySide6.
• Importance of dry-run defaults, centralised key release, and local-only processing for safety and privacy.
• Modular design that separates vision, decision making, and keyboard effects.

Conclusion

GesturePlay AI successfully demonstrates a complete pipeline from webcam hand tracking to real-time game control. By combining MediaPipe landmark detection, geometric and optional ML gesture recognition, careful smoothing, and a safety-first keyboard layer, the system provides an immersive alternative to traditional controllers. The modular architecture, dry-run default, and local processing make it both educational and practical. With good lighting and proper calibration, it offers a clear and engaging demonstration of applied computer vision for interactive applications.

— End of Report —